

Tips & tricks

Conditional output

Use `calepin-target` when a small piece of Typst should change between HTML and paged output:

```
#let target = sys.inputs.at("calepin-target", default: "paged")

#if target == "html" [
  This appears only in HTML.
] else [
  This appears in PDF, SVG, and PNG output.
]
```

Using codly with executed chunks

`codly` installs its own `show raw: rules`. `Calepin` wraps echoed source in a labeled block **around** that `raw`, so without an override you get `codly`'s frame nested inside `Calepin`'s box.

One `show` rule fixes it. This is a complete document, using the default `Calepin` theme:

```
#import "../calepin/calepin.typ" as calepin
#import "@preview/codly:1.3.0": *
#import "@preview/codly-languages:0.1.1": *

#show: calepin.document
#show: codly-init
#codly(languages: codly-languages)

// Hand the code to codly, and drop Calepin's own box from around it.
#show <calepin-input>: it => it.body

#calepin.setup(echo: true)

```python
import math
print(f"circumference: {2 * math.pi:.4f}")
```
```

Rendered result:

Open the rendered PDF.

Chunk output is untouched: it is not a `raw` element on the paged target, so `codly` never reaches it. The override reconstructs from `it.body` rather than re-emitting it; re-emitting would nest your rule around `Calepin`'s default instead of replacing it. See [Styling chunks](#) for the other labels and the full explanation.

Setting `theme = "typst"` removes `Calepin`'s chrome everywhere at once, with no `show` rules at all. Chunks still execute either way.